

The Mind Stone

MIRAGE 2.0 — Reverse Engineering — Walkthrough

CATEGORY	PORT	FILES
Reverse Engineering / Self-Modifying Code	1711/tcp	mind_stone_final (statically linked, stripped)
VULNERABILITY	STATUS	
N/A — JIT-decoded validator reversal	Solved	

Step 1 — First Contact

Connecting shows a “Vision Core” recovery terminal with three options: Inspect telemetry, Attempt stone recovery, Disconnect. Option 1 prints a wall of flavor text, including one line worth noting immediately: “STATE: SELF-MODIFYING CORE.” That’s not just theming — taken literally, it’s telling us the binary rewrites its own code at runtime, which means static disassembly alone won’t show the real check; whatever validates the input is assembled or decoded into memory only after the program is already running.

Option 1’s telemetry also includes a long hex string labeled TRACE. It’s tempting to assume this is a key or a hash to reverse — but nothing about it is confirmed useful yet. It goes on a list of “things to check later if the main path stalls,” not the first thing to chase.

Step 2 — A Naive First Attempt, and Why It Fails

Option 2 asks for a “stone recovery value” and rejects a test string with “incorrect lattice length” — so the check cares about length before content. The obvious next move for a reversing challenge is to attach gdb or run it under strace and step through what happens on option 2. Doing exactly that here is instructive: the binary still runs, still accepts input, but instead of the real result it prints “[!] STONE INTEGRITY FAILURE” / “Observer detected...” followed by what looks like a flag: MIRAGE{th3_m1nd_st0n3_b3l0ngs_t0_th4n0s}.

NOTE: That flag is a **decoy**. It sits in plaintext in the binary’s .rodata section specifically so that strings or a traced run will surface something flag-shaped and plausible-sounding — the whole point being to make a traced/debugged run *look* successful so a participant stops there. The giveaway is the “Observer detected” message itself, and the fact that the path only appears once a debugger or tracer is actually attached.

Step 3 — Reversing Without a Debugger

Static analysis of what triggers that decoy path (rather than dynamic tracing) shows two checks running before anything else on option 2: reading /proc/self/status for a non-zero TracerPid, and timing a 90,000-iteration LCG busy loop bracketed by CLOCK_MONOTONIC_RAW reads, failing if it took over roughly 20ms. Either one diverts to the decoy branch. This means the real logic has to be studied statically, and tested only by running the binary completely natively — no breakpoints, no single-stepping, not even a slow VM.

Length confirmed earlier (exactly 17 characters) plus “self-modifying core” from Step 1 together point at one specific pattern in the disassembly: a block that mmap's a fresh page, writes decoded bytes into it in a loop, then mprotect's it executable and calls into it. Finding that block and reading the decode loop shows a simple rolling keystream: `state = 0xd15ea5e5` initially, then `state = state * 0x19660d + 0x3c6ef35f` each step, combined with a byte offset that increases by `0x3d` per iteration — decoding a 349-byte embedded blob into the new executable page.

Step 4 — Reading the Decoded Code, Not Just the Decoder

Rather than letting the decoded page execute and trying to observe its effect indirectly, the decode loop's logic can simply be re-run offline in Python against the same embedded bytes to reconstruct exactly what code gets produced. Doing that reveals 17 small repeating blocks, one per expected input byte:

```
movzx al, [rdi]
inc   rdi
xor   al, X_i
add   al, Y_i
cmp   al, Z_i
jne   fail
```

Each position's (X, Y, Z) triple is a different embedded constant, but the shape of the check is identical and independent per byte — which means each character of the answer can be solved on its own, with simple algebra, rather than needing to brute-force all 17 positions together: `input[i] = X_i XOR ((Z_i - Y_i) & 0xFF)`. Running that formula against all 17 extracted constant triples reconstructs the entire expected string with no guessing.

Step 5 — Running It

```
$ nc <host> 1711
VISION@CORE> 2
[INPUT] Stone recovery value: MINDSTONE-TRUE-42
```

FLAG

```
MIRAGE{v1s10n_f0und_th3_m1nd_st0n3!_42}
```

Takeaway

When a target has visible anti-debug checks, the first useful experiment is often the “wrong” one done deliberately: attach a debugger anyway and see exactly what changes. A distinct, worse-looking result under tracing (here, an explicit “Observer detected” branch) confirms both that the check exists and roughly where it lives, and warns you not to trust whatever that branch prints as the real answer.